컨테이너 안의 GPU

무엇이 일어나고 어디서 깨지는가

AWSKRUG 컨테이너 소모임 · 2026-05-29 · 송이레

소개

- 송이레(MLOps Engineer)
- StradVision(2026.03.~)
- AEWS 4기 참여
- https://sirzzang.github.io

목차

메커니즘 · 깨질 자리

Chapter 1. GPU 코드 실행 계층
Chapter 2. 컨테이너 환경에서의 GPU
Chapter 3. 쿠버네티스 환경에서의 GPU

사례 · AWS 매핑 · 결론

Chapter 4. 사례 탐구
Chapter 5. 플랫폼의 책임 경계
Chapter 6. AWS 레퍼런스
Chapter 7. 결론

범위

다룬다

1. **컨테이너 안 GPU 코드 실행 메커니즘**
   NVIDIA GPU 한정 · 호스트 ↔ 컨테이너 ↔ H/W 흐름
2. **깨질 수 있는 지점 + 실제 사례 1건**
   PyTorch 분산 학습에서 NCCL collective 가 깨진 결
3. **AWS 레퍼런스**
   AWS에서 참고할 수 있는 것들

다루지 않는다

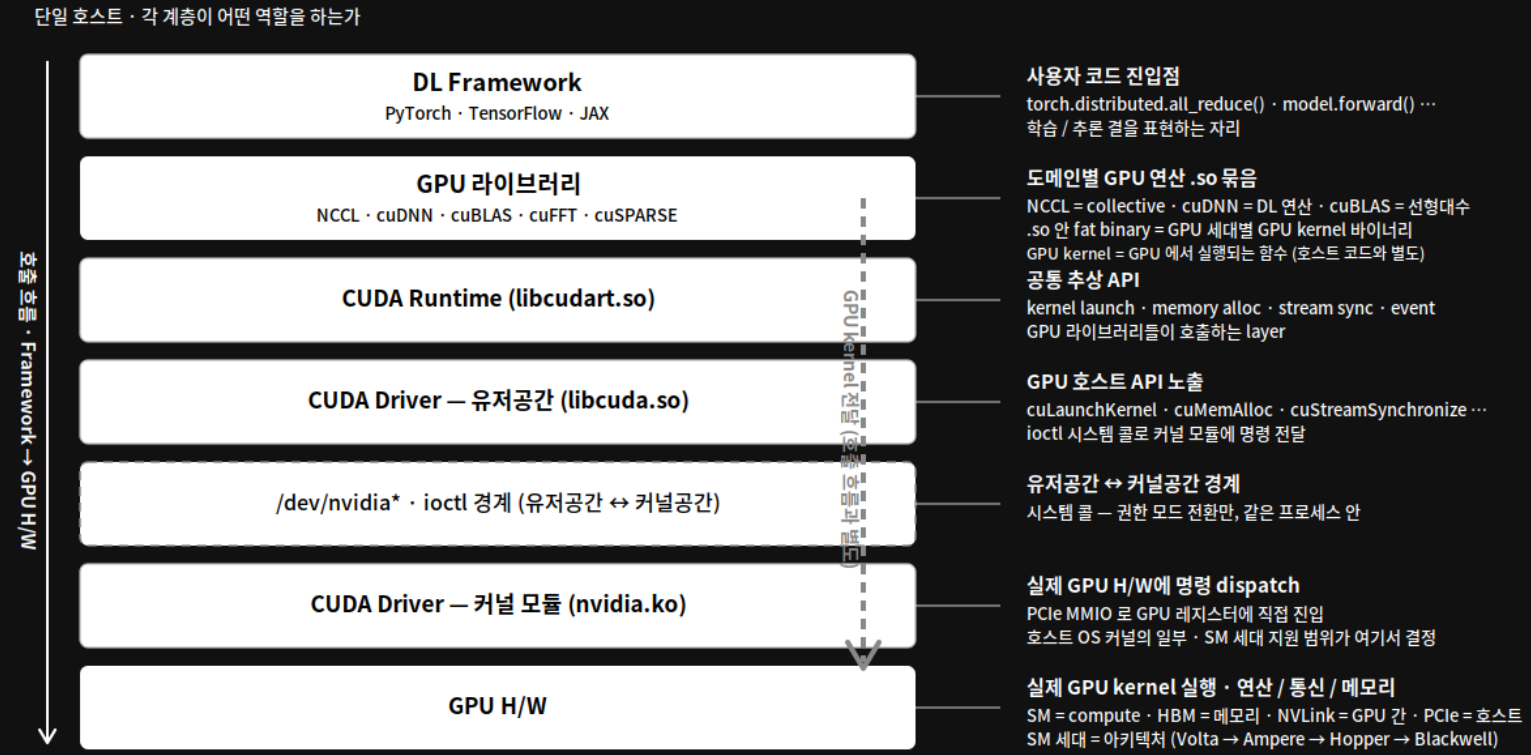
1. **GPU Operator 설치 / 운영 가이드**
   메커니즘은 다루지만 운영 가이드는 밖
2. **GPU 인프라 장애 트러블슈팅**
   사례는 1건만 (★ 빌드 타임 자리)
3. **플랫폼 엔지니어링 워크플로**
   IDP / DX / 셀프서비스 결 관점
4. **NVIDIA 아닌 가속기**
   Trainium · Inferentia · ROCm · oneAPI
5. **GPU H/W 내부 메커니즘**
   SM · HBM · Tensor Core · NVLink 내부
6. **GPU 네트워킹 상세 + 인프라 스택**
   EFA · GPUDirect RDMA · NCCL transport · libfabric

Chapter 1

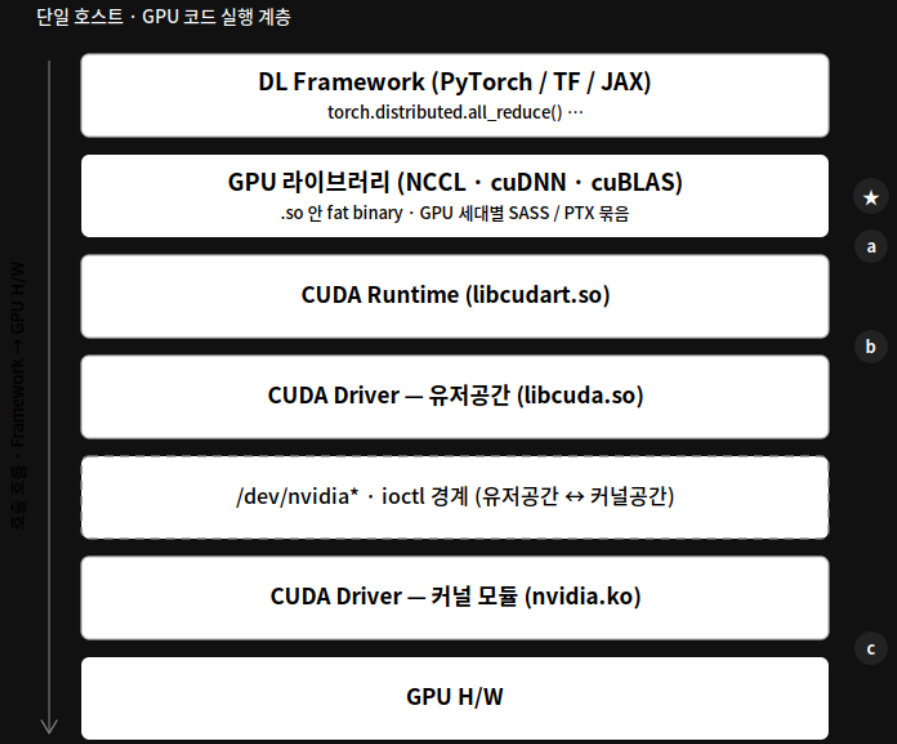
GPU 코드 실행 계층

단일 호스트에서 GPU 코드가 어떻게 실행되는가

단일 호스트 GPU 스택


단일 호스트 · 각 계층이 어떤 역할을 하는가
DL Framework
PyTorch · TensorFlow · JAX
GPU 라이브러리
NCCL · cuDNN · cuBLAS · cuFFT · cuSPARSE
CUDA Runtime (libcudart.so)
CUDA Driver — 유저공간 (libcuda.so)
/dev/nvidia* · ioctl 경계 (유저공간 ↔ 커널공간)
CUDA Driver — 커널 모듈 (nvidia.ko)
GPU H/W
호출 흐름 · Framework → GPU H/W
GPU kernel 전달 (호출 흐름과 별도)
사용자 코드 진입점
torch.distributed.all_reduce() · model.forward() …
학습 / 추론 결을 표현하는 자리
도메인별 GPU 연산 .so 묶음
NCCL = collective · cuDNN = DL 연산 · cuBLAS = 선형대수
.so 안 fat binary = GPU 세대별 GPU kernel 바이너리
GPU kernel = GPU 에서 실행되는 함수 (호스트 코드와 별도)
공통 추상 API
kernel launch · memory alloc · stream sync · event
GPU 라이브러리들이 호출하는 layer
GPU 호스트 API 노출
cuLaunchKernel · cuMemAlloc · cuStreamSynchronize …
ioctl 시스템 콜로 커널 모듈에 명령 전달
유저공간 ↔ 커널공간 경계
시스템 콜 — 권한 모드 전환만, 같은 프로세스 안
실제 GPU H/W에 명령 dispatch
PCIe MMIO 로 GPU 레지스터에 직접 진입
호스트 OS 커널의 일부 · SM 세대 지원 범위가 여기서 결정
실제 GPU kernel 실행 · 연산 / 통신 / 메모리
SM = compute · HBM = 메모리 · NVLink = GPU 간 · PCIe = 호스트
SM 세대 = 아키텍처 (Volta → Ampere → Hopper → Blackwell)

GPU 코드 실행의 두 축



두 축이 동시에 통과해야 GPU 코드가 돈다

① 런타임 호환

인접 레이어 인터페이스 계약 · 실행 시 검증

- **a** — **라이브러리 ↔ CUDA Runtime ABI**
  동적 링크 시 ABI 호환 · 정적 링크 시 해소
- **b** — **Driver CUDA cap ≥ Runtime CUDA**
  forward compatibility (CUDA 의 약속)
- **c** — **Driver 가 GPU 아키텍처 지원**
  nvidia.ko 가 SM 세대를 안다

② 빌드 타임 제약

라이브러리 .so 내부 · 빌드 시점에 고정

- ★ — **.so 안 fat binary 의 대상 GPU 아키텍처용 SASS 또는 호환 PTX 가 빌드 시점에 있어야 함**
  빌드 nvcc 가 모르는 SM 세대는 들어갈 수 없음
  런타임에 추가 / 변경 불가

두 축은 직교

한쪽이 통과해도 다른 쪽이 깨지면 실행 불가

a · b · c = 런타임 호환 (인터페이스 계약)
★ = 빌드 타임 제약 (.so 내부)

Chapter 2

컨테이너 환경에서의 GPU

스택은 그대로, 필요한 것만 호스트에서 빌려 온다

컨테이너 / 호스트 분리



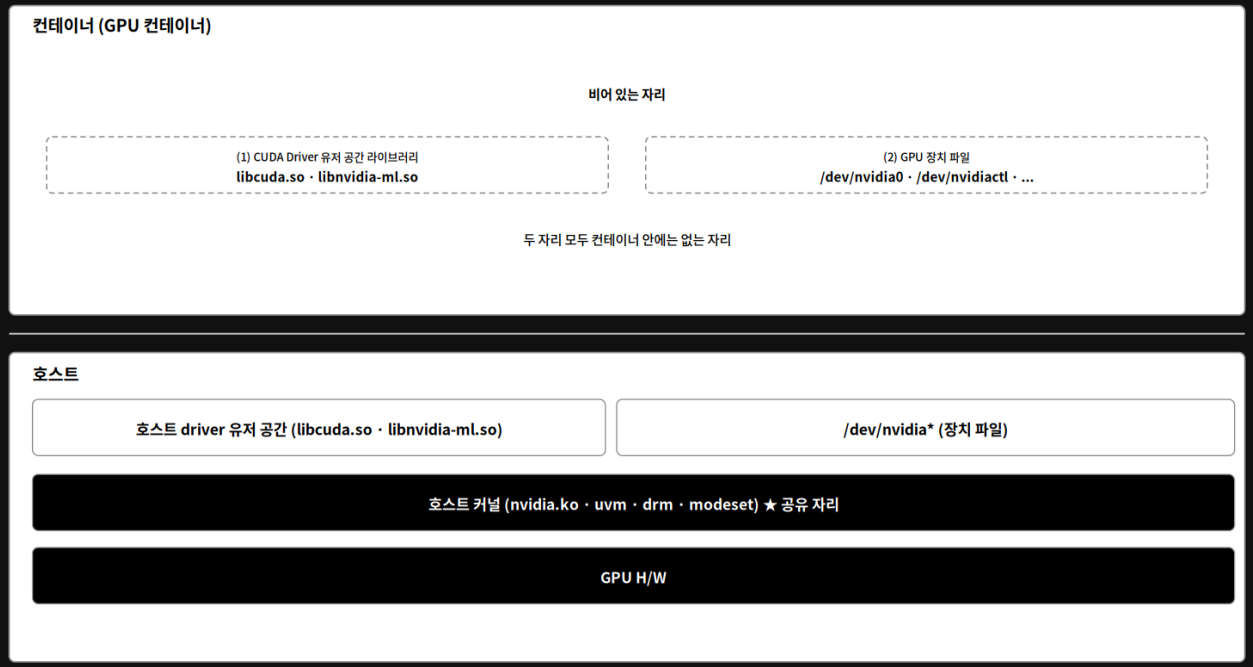
호스트는 GPU 와 커널을 가지고 있다. 다음 단계 — 컨테이너 안에는 무엇이 필요한가?

컨테이너 책임 자리는 어디부터인가?



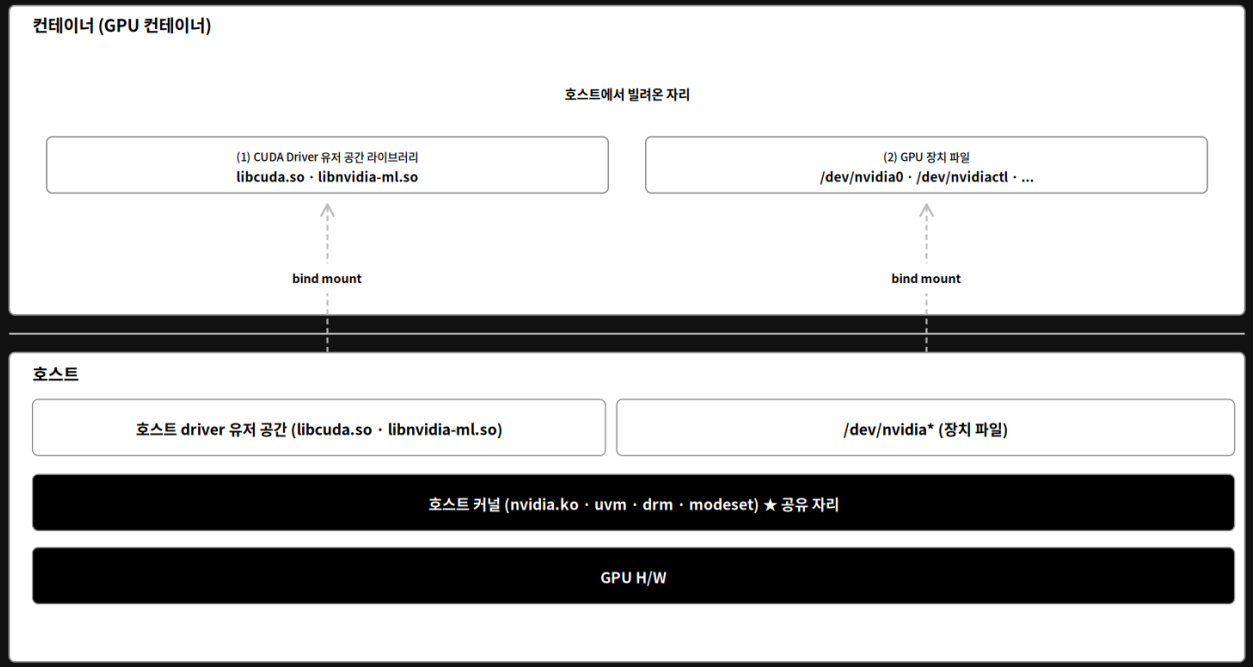
호스트 커널은 공유 — 컨테이너는 유저 공간만 책임진다. 다음 단계 — 정확히 무엇이 비어 있는가?

정확히 무엇이 비어 있는가



두 자리 다 호스트에는 이미 있다. 다음 단계 — 어떻게 가져오나?

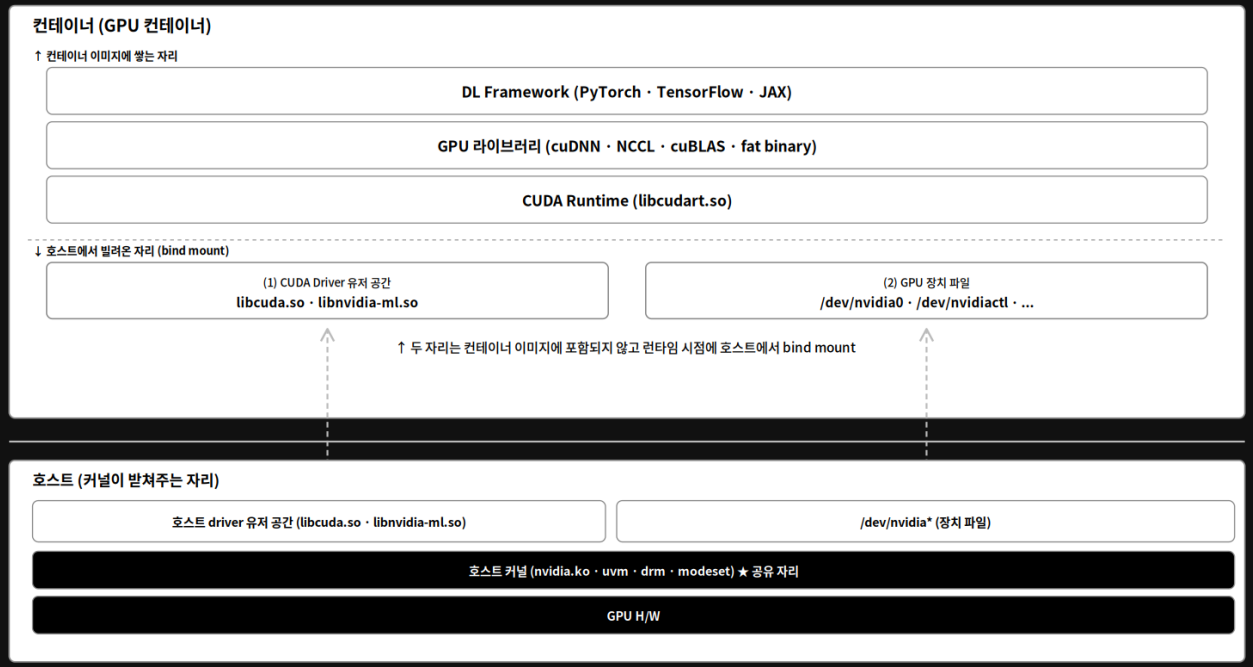
bind mount 로 호스트에서 빌려온다



호스트에서 빌려옴 — 두 비어 있던 자리가 채워졌다. 누가 bind mount 를 해 주는가는 다음 자리.

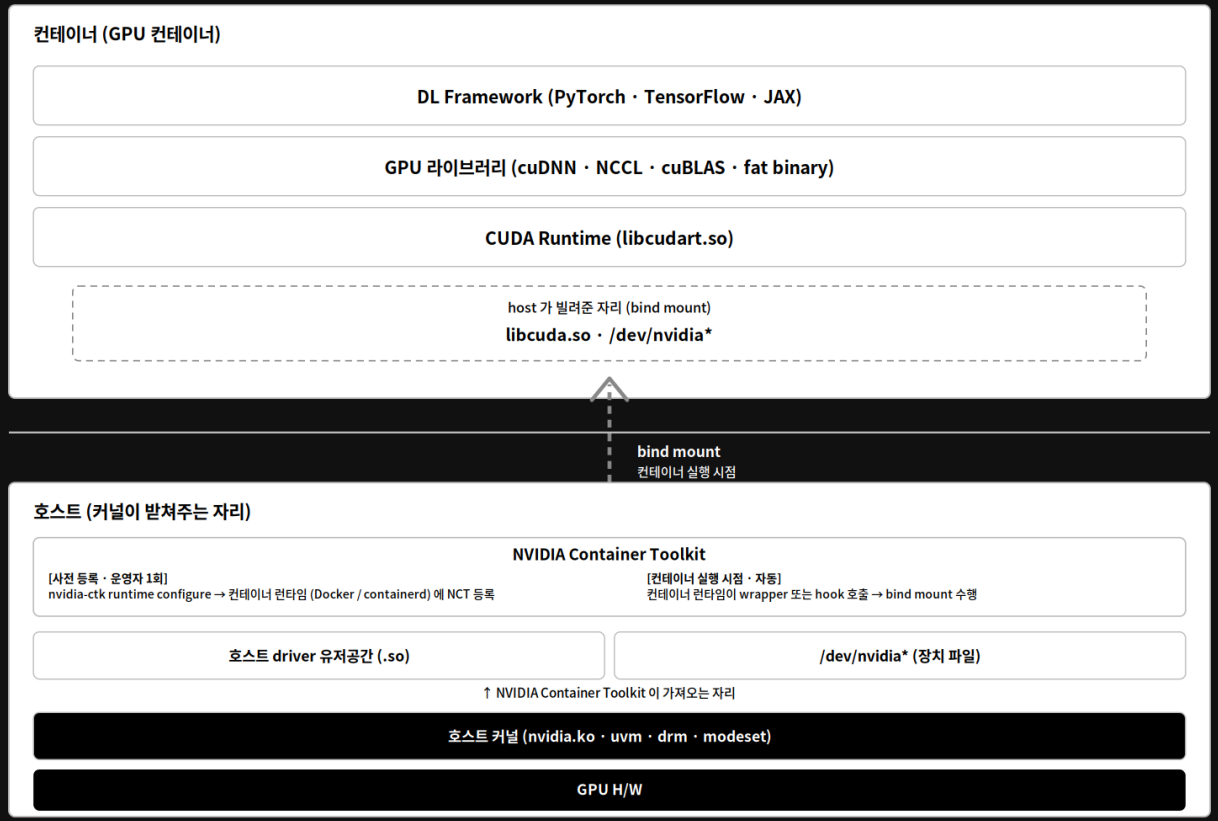
그 위는 컨테이너 이미지에 그대로 쌓는다

nvidia/cuda:11.8.0-**devel**-ubuntu22.04
nvidia/cuda:11.8.0-**runtime**-ubuntu22.04
nvidia/cuda:11.8.0-**base**-ubuntu22.04

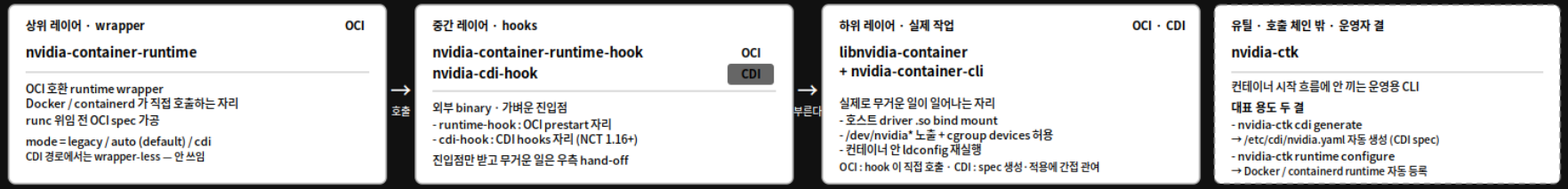


컨테이너 이미지 스택 + 호스트에서 빌려온 자리 = 완전한 GPU 컨테이너. 누가 bind mount 를 해 주는가?

NVIDIA Container Toolkit — 호스트와 컨테이너의 다리



(참고) NVIDIA Container Toolkit — 구성 요소 + 두 통합 경로



위 호출 체인은 OCI 경로 결 · CDI 경로에서는 wrapper 와 runtime-hook 이 빠지고 컨테이너 런타임이 spec 직접 처리 + cdi-hook 만 호출

두 통합 경로 · 컨테이너 런타임이 NCT 묶음을 어떻게 호출하는가

통합 경로 (1) · 레거시 · 현재 default

OCI prestart hook

OCI runtime spec 의 prestart 자리에 외부 program 한 번 끼움

호출 체인

```
Docker / containerd / kubelet
▼
nvidia-container-runtime
  | OCI spec 에 prestart hook 항목 주입
▼
runc
  | prestart 단계 진입 → hook binary 실행
▼
nvidia-container-runtime-hook
  | nvidia-container-cli configure ... 호출
▼
libnvidia-container + nvidia-container-cli
  | bind mount + cgroup + ldconfig 재실행
```

사용자 결: `docker run --gpus all` · `NVIDIA_VISIBLE_DEVICES=all`

사전 등록: `nvidia-ctk runtime configure --runtime=docker` → 런타임에 nvidia-container-runtime 등록

한계: vendor 마다 hook binary + 런타임 설정이 따로 — 가속기 다종 노드에서 어려움

통합 경로 (2) · vendor-neutral 표준 · 현재 권장

CDI (Container Device Interface)

vendor 측 spec 한 장을 컨테이너 런타임이 직접 읽어 OCI spec 에 반영

호출 체인 (wrapper-less direct CDI)

```
Docker / containerd / kubelet
  | --device nvidia.com/gpu=all 또는
  | cdi.k8s.io/<name> annotation
▼
Container Runtime
  | containerd 1.7+ / Docker 25+
  | /etc/cdi/nvidia.yaml spec 읽기
  | spec 안 mount · device · env 를 OCI spec 에 반영
▼
runc
  | 필요 시 hooks 단계에서
  | nvidia-cdi-hook
  | 실행 (ldconfig 재실행 / 심볼릭 링크 생성 등)
```

사용자 결: `docker run --device nvidia.com/gpu=all` · CDI device 이름

사전 등록: `nvidia-ctk runtime configure ... --cdi.enabled` + `nvidia-ctk cdi generate`

장점: vendor-neutral spec — 한 노드 안 GPU / RDMA / FPGA 한 결로 통합

Chapter 3

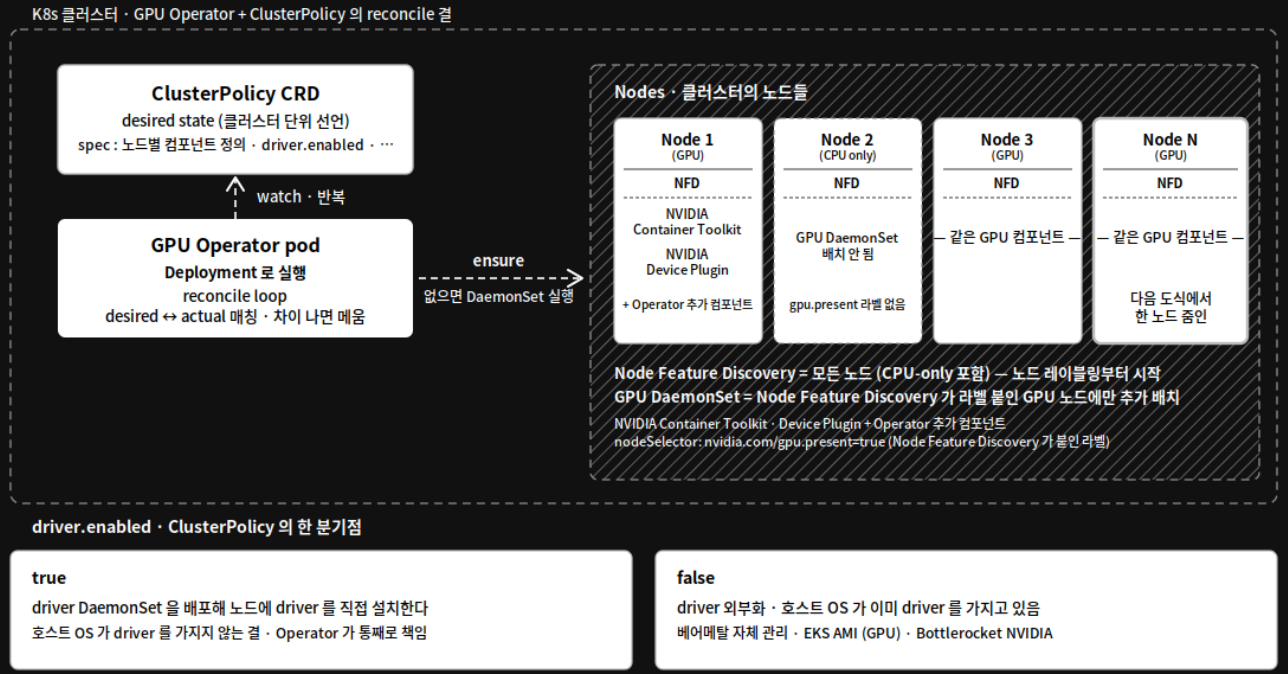
쿠버네티스 환경에서의 GPU

쿠버네티스가 GPU에 한 겹 더 올리는 것

K8s에서 GPU를 다루는 방법

- **K8s는 GPU를 모른다** — 표준 Device Plugin 인터페이스로 vendor가 채운다
- **NVIDIA Device Plugin** — NVIDIA의 K8s 표준 Device Plugin 구현
  - DaemonSet으로 배포되어 노드 GPU 자원을 kubelet에 보고 → nvidia.com/gpu: N
  - 전제 — 노드에 NVIDIA driver, NVIDIA Container Toolkit (챕터 2)
- (참고) DRA(Dynamic Resource Allocation) — K8s 1.34 GA, Device Plugin 개선
  - 개수 advertise → 속성 기반 조밀한 요청
  - ResourceClaim / DeviceClass — 클러스터 관리자 / 운영자 / 사용자(개발자) 계층 분리
- **GPU Operator** — Device Plugin 한 컴포넌트를 넘어, 노드 자동화 묶음 매니저
  - 작동 전제(driver · NVIDIA Container Toolkit · Device Plugin) 한 묶음으로 설치
  - 그 위 운영·관리 컴포넌트(NFD · GFD · DCGM · MIG manager · Validator 등) 설치
  - ClusterPolicy CRD로 클러스터 차원의 desired state 선언 및 GPU Operator Pod로 reconcile
  - 주로 Helm Chart로 배포되며, 환경에 따라 일부 컴포넌트 설치를 비활성화할 수 있음
- (참고) 그 위에 얹는 공유 메커니즘 — MIG · Time-Slicing · MPS

K8s 클러스터 — GPU Operator + ClusterPolicy reconcile



driver.enabled · ClusterPolicy 의 한 분기점

true

driver DaemonSet 을 배포해 노드에 driver 를 직접 설치한다
호스트 OS 가 driver 를 가지지 않는 경 · Operator 가 통째로 책임

false

driver 외부화 · 호스트 OS 가 이미 driver 를 가지고 있음
베어메탈 자체 관리 · EKS AMI (GPU) · Bottlerocket NVIDIA

GPU Operator 컴포넌트

NVIDIA Container Toolkit
컨테이너에 GPU 노출

NVIDIA Device Plugin
kubelet 에 GPU advertise

Node Feature Discovery
노드 레이블링 · 모든 노드

GFD (GPU Feature Discovery)
GPU 세부 레이블 · 모델 · VRAM · CUDA ver

DCGM + DCGM Exporter
GPU 메트릭 수집 · Prometheus 노출

Validator
설치 검증 · 노드 상태 업데이트

MIG Manager
A100 / H100 MIG 파티셔닝

VGPU Manager
vGPU 라이선스 · driver 관리

Sandbox Device Plugin
vGPU / GPU passthrough 환경용

kubectl get pods -n gpu-operator

```
nvidia-driver-daemonset-xxx
nvidia-container-toolkit-daemonset-xxx
nvidia-device-plugin-daemonset-xxx
nvidia-dcgm-xxx
nvidia-dcgm-exporter-xxx
gpu-feature-discovery-xxx
node-feature-discovery-xxx
gpu-operator-validator-xxx
```

노드 줌인 — User Pod + DaemonSet + 호스트



Chapter 4

사례 탐구

분산학습 GPU Pod에서 NCCL 이 깨졌다

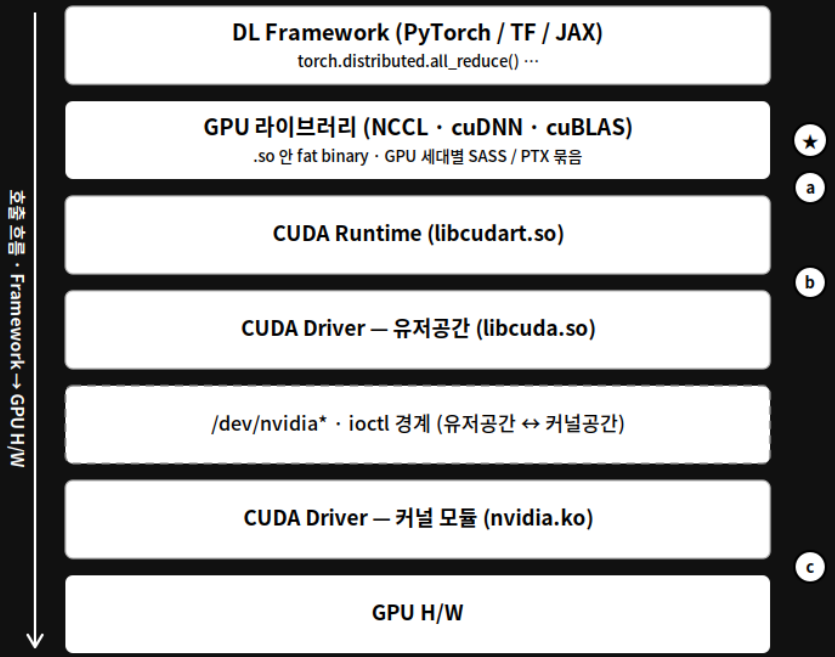
증상

```
$ python train.py


[rank0]: torch.distributed.DistBackendError: NCCL error
[rank0]: ...
[rank0]: CUDA error: an illegal memory access was encountered
[rank0]: CUDA kernel errors might be asynchronously reported at some other API call,
[rank0]: so the stacktrace below might be incorrect.
```

- PyTorch 분산 학습 Pod — **학습 시작 직후 NCCL collective op에서 fail**
  - 환경: K8s · RTX 5090 · driver 580.126.09 · CUDA 12.8
  - 학습 이미지: MLOps 플랫폼 베이스 이미지 + ML 필요 스택
- **CUDA error** — 인프라 문제 < **GPU 컨테이너 개발 환경 문제** 의심
- 어디서 깨지는지 챕터 1의 내용을 근거로 살펴 보자

분석 — 런타임 호환 제약 모두 통과



본 사례 — 런타임 호환 (a)(b)(c) 확인 사실

(a) 라이브러리 ↔ CUDA Runtime ABI — ✓문제 아님

```
$ readelf -d libnccl.so.2 | grep NEEDED
```

→ libcudart.so 없음 (정적 링크 — 빌드 타임에 굳음)

(b) Driver CUDA cap ≥ Runtime CUDA — ✓문제 아님

```
nvidia-smi (13.0) ≥ torch.version.cuda (12.8)
```

→ driver capability 가 번들 runtime 을 cover (forward compatibility 충족)
번들 wheel은 시스템 nvcc 안 씀 — torch.version.cuda 가 실제 runtime

(c) Driver ↔ GPU 아키텍처 (nvidia.ko · SM 세대) — ✓문제 아님

```
$ nvidia-smi
```

→ GPU 정상 인식 · driver 555+ (sm_120 지원 시작 이후)

남은 자리 — ★ 빌드 타임 제약

★ **라이브러리 .so 안 fat binary 의 SASS / PTX**
대상 GPU 아키텍처용 GPU kernel 이 빌드 타임에 포함됐는가

런타임 호환 세 자리는 다 통과했지만 — 첫 GPU kernel 론칭이 깨졌다.
두 축이 직교 (챕터 1 회수) — 런타임 호환이 통과해도 빌드 타임이 깨지면 실행 불가.

NCCL 라이브러리 (.so) 의 fat binary 를 까보자

진단 시간선 — 첫 한 시간: NCCL_DEBUG=INFO · 다음 단: nvidia-smi / torch.version.cuda 값 비교 · 그 다음 단: 번들 NCCL 빌드 CUDA 자리

분석 — 빌드 타임 제약 확인

libnccl.so.2 (ELF 파일)
번들 NCCL · PyTorch wheel 의존성으로 pip install

(a) 호스트 측 진입점
CPU 가 실행할 C/C++ 코드 — ncclAllReduce() 등의 진입 심볼
CUDA Runtime 정적 링크 (libcudart_static.a 가 .so 안에 통째로 포함)

(b) fatbinary 영역 — 별도 ELF 섹션
여러 GPU 세대용 GPU kernel 바이너리가 함께 포함

SASS · GPU 아키텍처별 네이티브 기계어

sm_70	sm_75	sm_80	sm_86	sm_89
Volta · V100	Turing · T4	Ampere · A100	Ampere · A10	Ada · L40

sm_90 (Hopper · H100) ← 번들 NCCL 빌드 CUDA 12.2 의 한계 — CUDA 12.2 nvcc 는 sm_90 까지만 안다 — sm_120 (Blackwell · 없음)

PTX · GPU 독립 중간 표현 (driver 가 JIT 컴파일 가능)

compute_90 PTX	compute_120 PTX
존재	없음 → JIT 불가

sm_120 SASS 없음 · compute_120 PTX 없음 → driver JIT 도 실패

(1) fat binary 진단

.so 안에 어떤 GPU 아키텍처 SASS 가 들어 있는가

```
$ cuobjdump -lelf libnccl.so.2 | grep sm_
```

본 사례 출력 — sm_120 (RTX 5090) 없음

```
elf.sm_70 elf.sm_75 elf.sm_80 elf.sm_86 elf.sm_89 elf.sm_90
```

(2) 번들 NCCL 빌드 CUDA 진단

왜 sm_120 이 빠졌는가 — 어떤 CUDA 버전으로 빌드되었는지 확인

```
$ python -c "import torch; print(torch.version.cuda)" → 12.2
```

CUDA 12.2 nvcc	→ sm_90 (Hopper) 까지만 안다	← 본 사례 번들 NCCL
CUDA 12.8 nvcc	→ sm_100 (B200) · sm_120 (RTX 5090) 추가	

12.2 nvcc 는 sm_120 인자 자체를 모름 — 빌드 시점에 모르는 아키텍처는 fat binary 에 들어갈 수 없다.
시스템 CUDA 를 갈아끼워도 · driver 를 올려도 · 환경변수를 만져도 — 빌드된 .so 안 SASS 는 새로 생기지 않는다.

★ 한 줄 결론

번들 NCCL .so 는 CUDA 12.2 빌드의 한계로 묶여 있다.
sm_120 cover 하려면 라이브러리 .so 자체를 새 빌드로 교체할 수밖에 없다.

결과

- 원인 확정: **빌드 타임 제약**
  - PyTorch wheel에 의해 자동으로 설치된 NCCL 라이브러리(nvidia-nccl-cu12 wheel)
  - 해당 라이브러리는 **CUDA 12.2** 빌드 — RTX 5090이 필요로 하는 sm_120 (Blackwell) 모름
- 해결: **학습 이미지 재빌드**
  - 시스템 CUDA 갈아끼워도 / driver 올려도 / 환경변수 만져도 — 빌드 결과 .so 안 SASS 는 새로 안 생김
  - 라이브러리 .so 자체를 운영 GPU 세대를 cover 하는 빌드로 교체
- 다른 사례 참고: Dev0ps[AEWS 4기] 9주차 - GPU on EKS 실제로 운영하면서 겪은 흔한 이슈들
  - 오늘 소개한 사례와 다른 결에서 깨진 사례 → 비교해 보시면 좋겠습니다 :)
- → 이런 사례들의 **재발은 어떻게 막아야 하는가**

Chapter 5

플랫폼의 책임 경계

플랫폼이 어디까지 흡수할 수 있는가

베이스 이미지에 어디까지 담을 것인가?

5 단 자리

①	**GPU Driver** nvidia.ko · libcuda.so	`$ apt install nvidia-driver-...` 호스트에 설치
	↑ 호스트 · ↓ 컨테이너	
②	**CUDA Toolkit** 컴파일러 + libcudart	`FROM nvidia/cuda:12.8-...`
③	**시스템 NCCL** apt install libnccl2	`RUN apt install libnccl2`
④	**DL Framework + 번들 NCCL** PyTorch wheel (cu128) · 번들 NCC	`RUN pip install torch`
⑤	**ML 모델 코드** 학습 / 추론 스크립트	`COPY src/ /app/`

베이스 이미지 점층 — 어디까지 미리 박을 것인가

Case 1	Case 2	Case 3 · 발표자 환경	Case 4
② 까지만 (CUDA base) ML 엔지니어 자율 최대	③ 까지 (시스템 NCCL 포함) 버전 정합 보장 시작	④ 까지 (Framework + 번들 NCCL) 플랫폼이 ④ 까지 책임	⑤ 까지 (전부 박음) 완전 빌드된 결 · CI 산출물
⑤ ML 코드 (ML 엔지니어 직접 박음)	⑤ ML 코드 (ML 엔지니어 직접 박음)	⑤ ML 코드 (ML 엔지니어 자율)	⑤ ML 코드
④ DL Framework (ML 엔지니어 직접 박음)	④ DL Framework (ML 엔지니어 직접 박음)	④ DL Framework + 번들 NCCL (PyTorch wheel)	④ DL Framework + 번들 NCCL
③ 시스템 NCCL (선택적)	③ 시스템 NCCL	③ 시스템 NCCL	③ 시스템 NCCL
② CUDA Toolkit	② CUDA Toolkit	② CUDA Toolkit	② CUDA Toolkit
호스트	호스트	호스트	호스트
① GPU Driver	① GPU Driver	① GPU Driver	① GPU Driver
베이스 이미지 결 ② 만 박힘	베이스 이미지 결 ② + ③ 박힘	베이스 이미지 결 ② + ③ + ④ 박힘	베이스 이미지 결 ② + ③ + ④ + ⑤ 박힘

ML 엔지니어 자율 (큼) → 플랫폼 책임 (큼)

발표자 환경 → Case 3 결로 ④ 까지 베이스 이미지에 담음

챕터 5 / 6 사례 (PyTorch 번들 NCCL 깨진 자리) 계기로 ④ 자리를 ML 엔지니어 결에서 플랫폼 결로 한 단 당겨 옴.
사내 blessed 베이스 이미지 + 번들 NCCL wheel pin · ⑤ 만 ML 엔지니어 자율로 남김

(참고) 재발 방지 접근법

#	접근법	개입 자리	성격	ML 자유도	핵심 비용 · 비고
A	**NGC 등 pre-validated 이미지** NVIDIA 가 빌드해 검증한 묶음	스택 전체 일괄	예방	낮음	NVIDIA 릴리스 cadence 종속 · 커스터마이징 제약 신형 GPU 세대 cover 자리는 NVIDIA 측 빌드 cadence가 결정
B	**플랫폼 blessed 이미지** PyTorch + 번들 NCCL 까지 사내 빌드	Framework + 코드	예방	낮음	빌드·검증 owner cost · 업그레이드마다 플랫폼 개입 AWS DLC = B 의 AWS 변형 (Base/Framework 두 단 분리, 챕터 8 Layer 4)
C	**시스템 NCCL 강제** 번들 제거 + apt + ldconfig	시스템 NCCL (+코드)	예방	높음	RPATH 우회 침습 · pip install torch 재실행 시 재설치 → CI 게이트 필수 한 자리를 닫으면서 다른 자리 (a) ABI 가 다시 열림
C'	**시스템 NCCL 설치만 (보험)** 발표자 환경 미적용	시스템 NCCL	보험	최대	단독으로는 정합 X (번들이 RPATH 로 먼저 로드) · B / E 조합 필요 번들 wheel 이 제거됐을 때의 폴백 자리만 의미 있음
D	**번들 NCCL wheel pin (B 변형)** nvidia-nccl-cu12 wheel 한 패키지만 갈아끼움	번들 NCCL wheel	예방	중간	PyTorch 자체 빌드 (전체 빌드) 자리 — 실제로는 wheel 한 패키지만 갈아끼움 발표자 환경: B (blessed 베이스) + D (번들 NCCL wheel pin) 하이브리드 — owner cost 한 단 낮춤

A~D 전 영역에 도입 가능한 횡단 게이트

E. 호환성 가이드
문서 · 경고
ML 엔지니어 자율 + 문서
사람 행동 의존 · 누락 시 재발

F. 빌드 타임 게이트
CI smoke test
이미지 빌드 후 분산 학습 1 epoch
production push 전 사후 검출

G. 런타임 게이트
startup assertion (fail-fast)
entrypoint 에서 버전 매트릭스 검증
첫 all_reduce 전 명확한 에러로 죽음

추가 도입: 플랫폼 측 호환성 매트릭스 게이트 관리

호스트 측 · GPU Driver 자리

노드별 driver 버전 게이트
클러스터 전체 노드의 driver 버전 매트릭스 관리
GPU Operator ClusterPolicy · 호스트 AMI

driver 별 CUDA 런타임 호환성 매트릭스
driver N 이 어떤 CUDA Runtime / fat binary 를 받쳐줄 수 있는가
forward compatibility · CUDA Compatibility Package 확인
신형 GPU 도입 전 driver / Runtime 매트릭스 사전 검증

컨테이너 측 · 컨테이너 이미지 스택 자리

GPU 세대별 fat binary cover 매트릭스
번들 NCCL / cuDNN / cuBLAS 가 sm_? 까지 cover 하는가
cuobjdump -lelf 로 라이브러리별 SASS 커버리지 검증

베이스 이미지 + wheel 버전 정합 게이트
blessed 베이스 이미지 + pin 된 wheel 버전 → 빌드 cadence 일관성 확보
CUDA Toolkit · 시스템 NCCL · 번들 NCCL 의 정합 매트릭스
신형 GPU 세대 도입 시 라이브러리 .so 교체 cadence 관리

Chapter 6

AWS 레퍼런스

AWS에서 배울 수 있는 것

AWS가 받쳐 주는 자리

사용자가 직접 다루는 영역 → AWS 가 가져가는 영역

단계 1 · 호스트 측
AMI / Bottlerocket NVIDIA

AWS 가 받쳐 주는 자리

호스트 driver 의존성을 OS 이미지가 받쳐 줌
사용자는 nvidia-driver 설치 / 버전 정합을 직접 안 다룸

대표 옵션

- Amazon Linux 2 GPU AMI (driver baked-in)
- Bottlerocket NVIDIA (k8s 전용 OS · driver baked-in)
- EKS Optimized AMI (GPU)

참고 · EKS Managed Node Groups → AMI rollout 까지 자동
참고 · EKS Auto Mode → 노드 프로비저닝까지 자동
참고 · driver 가 받쳐 줘도 컨테이너 안 의존성은 그대로 남음

단계 2 · 컨테이너 이미지
AWS Deep Learning Containers

AWS 가 받쳐 주는 자리

AWS 가 직접 빌드해 둔 베이스 이미지
ECR 에서 pull · 직접 빌드 / 검증 비용 0

대표 옵션

- DLC Base (CUDA Toolkit + 공용 라이브러리)
- DLC Framework (DL Framework + 분산 학습 라이브러리)
- ECR Public 에서 pull · 사용 즉시 가능

참고 · 여기서부터 사용자는 CUDA Runtime 만 신경 써서 선택
참고 · DLC 안 Framework 빌드 = AWS 가 결정한 빌드 cadence
참고 · ML 코드는 여전히 사용자 자리

단계 3 · 컨테이너 메커니즘
SageMaker

AWS 가 받쳐 주는 자리

컨테이너 메커니즘 자체가 추상화 — 사용자가 직접 풀 일 없음
컨테이너 / 노드 자리는 안 보이고 Job 정의만 다룸

대표 옵션

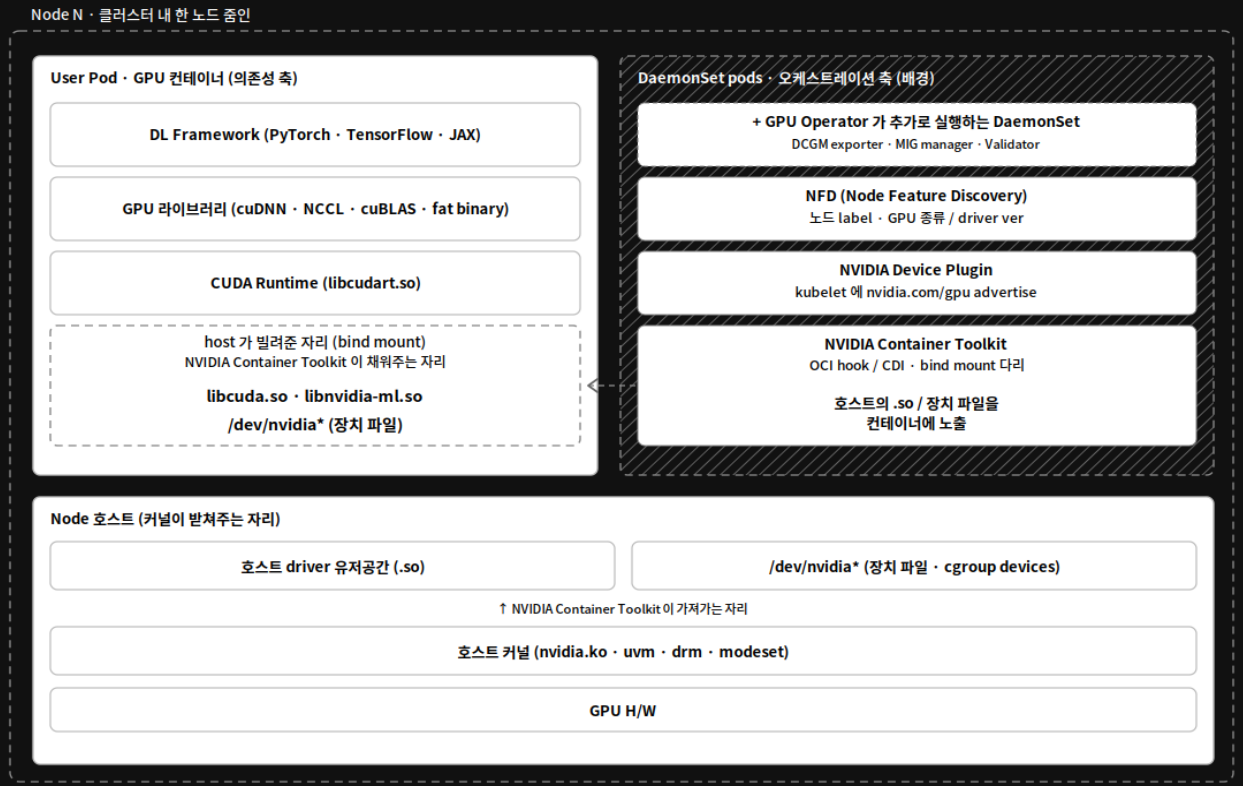
- Training Jobs (학습 단발성 실행)
- HyperPod (분산 학습 클러스터 추상화)
- Built-in Algorithm · DLC · BYOC (자체 이미지) 도 선택 가능

참고 · BYOC 로 자체 이미지를 가져오면 다시 사용자 자리로 돌아옴
참고 · EKS 와 다른 책임 모델 — 노드 / Pod 자리 안 보임
참고 · 추상화 비용 = 디버깅 / 커스터마이징 자유도 줄어듦

오늘 다루지 않는 AWS 자리

- EC2 raw · DLAMI — IaaS / 베어 EC2 운영 자리 (오늘 발표 대상 아님)
- EKS Self-managed — 노드 lifecycle 직접 관리 자리 (위 AMI / DLC 도구 자체는 동일하게 사용 가능)
- Bedrock — 완전 API 추상화 자리 (컨테이너 / 노드 자리 안 노출, 발표 자료 밖)
- ECS / Batch — 다른 컨테이너 오케스트레이션 자리 (같은 도구 결로 풀이 가능)

AWS 매핑 — GPU 개발환경 스택 매칭



AWS 매핑 — 스택 매칭

자료 기반 매핑 · 정답 아님

DLC Framework (PyTorch · TF · JAX 등)

DL Framework + GPU 라이브러리
+ TransformerEngine · flash-attn · triton
(학습 / 추론 가속 도구 — Framework 결로 함께 빌드)

DL Framework + GPU 라이브러리 + CUDA Runtime 자리

DLC Base

FROM 공통 BASE (단일 image)
CUDA Runtime + GPU 라이브러리
(cuDNN · 시스템 NCCL 등 — 빌드 시 기본 포함)
aws-ofi-nccl · EFA · OpenMPI

한 묶음으로 같은 nvcc cycle 빌드 (추정)
DLC Framework 와 빌드 cadence 정합

CUDA Toolkit + 시스템 NCCL 자리

Bottlerocket NVIDIA

immutable host OS
driver + Container Toolkit + Device Plugin baked-in

↔ GPU Driver 자리 (호스트)

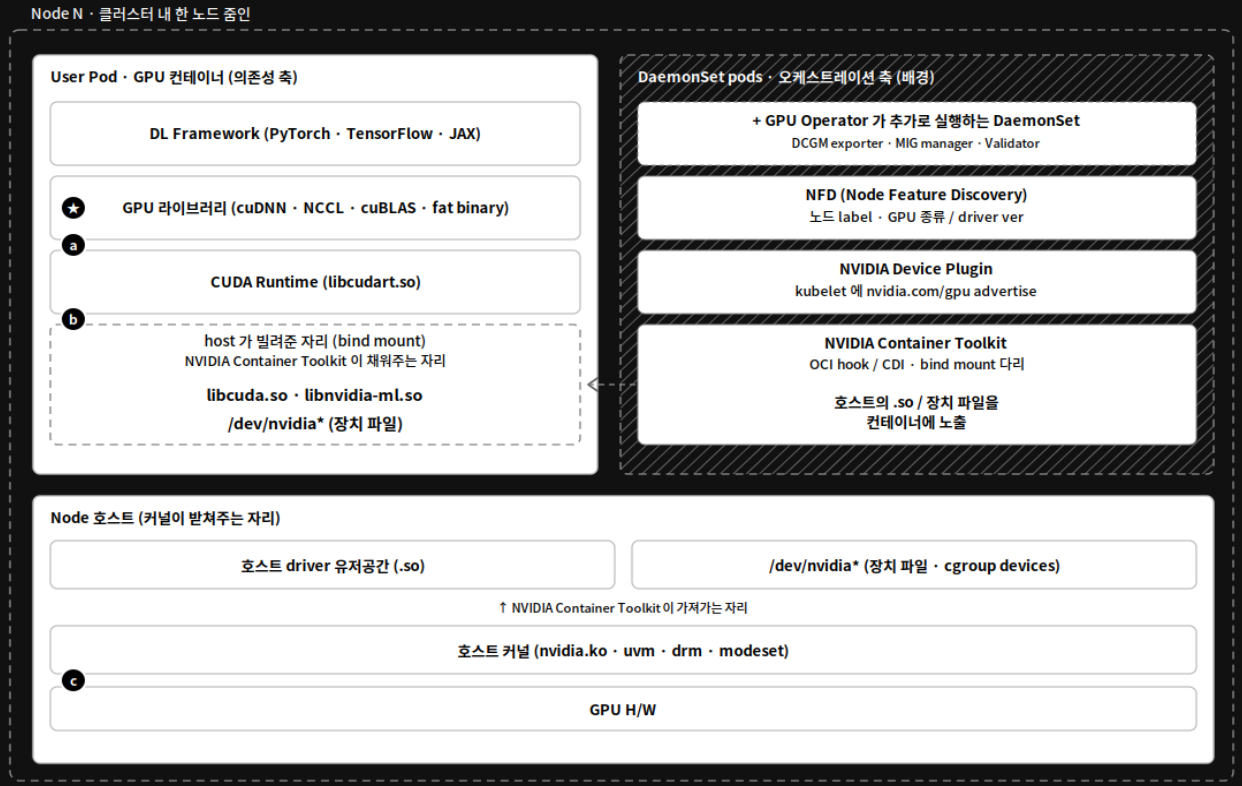
EKS AMI (GPU)

AL2023 NVIDIA
driver + NVIDIA Container Toolkit baked-in

↔ GPU Driver 자리 (호스트)

별책 EFA device plugin · Capacity Block ML

AWS 매핑 — 깨질 자리 차단



전제 — 같은 시점의 AMI + DLC 조합
자기 이미지 BYO · AMI pin · 버전 부정합 결로 박으면
여전히 깨질 수 있음

a 라이브러리 ABI

DLC Framework 이미지 안에서
CUDA Runtime · GPU 라이브러리 · DL Framework 함께 빌드 · 테스트
→ 이미지 내부 호환 보장

b driver CUDA cap ≥ container CUDA runtime

AMI driver 가 DLC CUDA runtime 을 보장하도록
AWS 가 같은 시점에 맞춰 릴리스

c driver ↔ GPU 아키텍처

AWS 가 인스턴스 라인업을 보장하는 AMI 릴리스

★ 빌드 타임 제약 (신형 GPU 세대 cover) — 추정

NVIDIA 가 신형 GPU 지원 CUDA Toolkit 릴리스
→ AWS 의 DLC 리빌드
→ 신형 GPU 라인업 cover
신형 인스턴스 GA 직후 일시 갭 가능

Chapter 7

결론

왜 이렇게 복잡하며, AWS에서 무엇을 배울 수 있는가

- **GPU 실행 환경은 복잡하다** — 여러 겹의 복잡함이 누적된 자리
  - GPU H/W: (오늘 다루진 않았으나) H/W 자체도 복잡함(SM, HBM, NVLink, PCIe, …)
  - GPU S/W 프로그래밍 스택: DL Framework → GPU 라이브러리 → CUDA Runtime → CUDA Driver → OS 커널 → GPU H/W
  - 컨테이너 환경: 컨테이너/호스트 분리 + NVIDIA Container Toolkit과 bind mount
  - K8s 환경: Device Plugin, GPU Operator에 의한 운영 자동화
- 컨테이너 안 GPU 코드 실행 측면에서**, 호스트와 컨테이너가 크게 다르지는 않다** → **깨졌을 때**는 런타임 호환 a·b·c + ★ 빌드 타임을 의심하자
- AWS에서 적절한 추상화를 배울 수 있다: H/W → GPU 소프트웨어 스택 → 컨테이너 환경 중 필요한 자리만 노출
  - 하드웨어 정합성 보장: DLAMI, EKS-Optimized AMI, Bottlerocket
  - 컨테이너 책임 경계의 이동: 플랫폼 컨테이너 책임 경계를 DLC Base, DLC Framework 로 분리하여 흡수
  - SageMaker(프로토콜), Bedrock(API) 등 다양한 수준에서의 옵션 제공

(참고) 정리 — K8s GPU 컨테이너 주의 사항

레벨	자리	기존 챕터 내용 매칭
인프라 노드 / 클러스터 자동화 자리	• GPU Operator chart 설치 / reconcile • NVIDIA Container Toolkit 설정 drift • 노드 GPU 인식 실패 • NVIDIA Device Plugin advertise 실패	**챕터 3 K8s 자리** Part A 에서 다룸
컨테이너 개발 환경 이미지 안 계층 호환성	★ **라이브러리 .so 안 SASS / PTX 포함 여부** fat binary 안에 운영 GPU 아키텍처용 GPU kernel 이 빌드 타임에 포함됐는가 → 라이브러리 빌드 시점에 nvcc 한 번이 결정하고 끝남 · 런타임 변경 불가	**챕터 1 ★** 빌드 타임 제약 .so 내부 fat binary
	a **라이브러리 ↔ CUDA Runtime ABI** 동적 링크 시 적용 · 정적 링크 시 자동 해소	**챕터 1 (a) ABI**
	b **Driver CUDA cap ≥ Runtime CUDA** forward compatibility · 호스트 driver 가 받아낼 수 있는 상한	**챕터 1 (b)** forward compatibility
	c **Driver ↔ GPU 아키텍처** nvidia.ko 가 GPU 의 SM 세대 (sm_120 등) 를 지원	**챕터 1 (c)** nvidia.ko · SM 세대

감사합니다